In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import datetime as dt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import seaborn as sns
```

In [2]:
```
# Load dataset
df = pd.read_csv('sales_data_sample.csv', encoding='unicode_escape')
```

In [3]:
```
df.head()
```

Out[3]:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDER
0	10107	30	95.70	2	2871.00	2/24
1	10121	34	81.35	5	2765.90	5/7
2	10134	41	94.74	2	3884.34	7/1
3	10145	45	83.26	6	3746.70	8/25
4	10159	49	100.00	14	5205.27	10/10

5 rows × 25 columns

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   ORDERNUMBER       2823 non-null   int64  
 1   QUANTITYORDERED   2823 non-null   int64  
 2   PRICEEACH         2823 non-null   float64
 3   ORDERLINENUMBER   2823 non-null   int64  
 4   SALES             2823 non-null   float64
 5   ORDERDATE         2823 non-null   object 
 6   STATUS            2823 non-null   object 
 7   QTR_ID            2823 non-null   int64  
 8   MONTH_ID          2823 non-null   int64  
 9   YEAR_ID           2823 non-null   int64  
 10  PRODUCTLINE       2823 non-null   object 
 11  MSRP              2823 non-null   int64  
 12  PRODUCTCODE       2823 non-null   object 
 13  CUSTOMERNAME      2823 non-null   object 
 14  PHONE             2823 non-null   object 
 15  ADDRESSLINE1      2823 non-null   object 
 16  ADDRESSLINE2      302 non-null    object 
 17  CITY              2823 non-null   object 
 18  STATE             1337 non-null   object 
 19  POSTALCODE        2747 non-null   object 
 20  COUNTRY           2823 non-null   object 
 21  TERRITORY         1749 non-null   object 
 22  CONTACTLASTNAME   2823 non-null   object 
 23  CONTACTFIRSTNAME  2823 non-null   object 
 24  DEALSIZE          2823 non-null   object 
dtypes: float64(2), int64(7), object(16)
memory usage: 551.5+ KB
```

In [5]:
```python
# Drop unnecessary columns
to_drop = ['ADDRESSLINE1', 'ADDRESSLINE2', 'STATE', 'POSTALCODE', 'PHONE']
df = df.drop(to_drop, axis=1)
```

In [6]:
```python
#Check for null values
df.isnull().sum()
```

Out[6]:
```
ORDERNUMBER           0
QUANTITYORDERED       0
PRICEEACH             0
ORDERLINENUMBER       0
SALES                 0
ORDERDATE             0
STATUS                0
QTR_ID                0
MONTH_ID              0
YEAR_ID               0
PRODUCTLINE           0
MSRP                  0
PRODUCTCODE           0
CUSTOMERNAME          0
CITY                  0
COUNTRY               0
TERRITORY          1074
CONTACTLASTNAME       0
CONTACTFIRSTNAME      0
DEALSIZE              0
dtype: int64
```

In [7]:
```
df.dtypes
```

Out[7]:
```
ORDERNUMBER           int64
QUANTITYORDERED       int64
PRICEEACH           float64
ORDERLINENUMBER       int64
SALES               float64
ORDERDATE            object
STATUS               object
QTR_ID                int64
MONTH_ID              int64
YEAR_ID               int64
PRODUCTLINE          object
MSRP                  int64
PRODUCTCODE          object
CUSTOMERNAME         object
CITY                 object
COUNTRY              object
TERRITORY            object
CONTACTLASTNAME      object
CONTACTFIRSTNAME     object
DEALSIZE             object
dtype: object
```

In [8]:
```
# Select numeric columns only
df_numeric = df.select_dtypes(include=['int64', 'float64'])

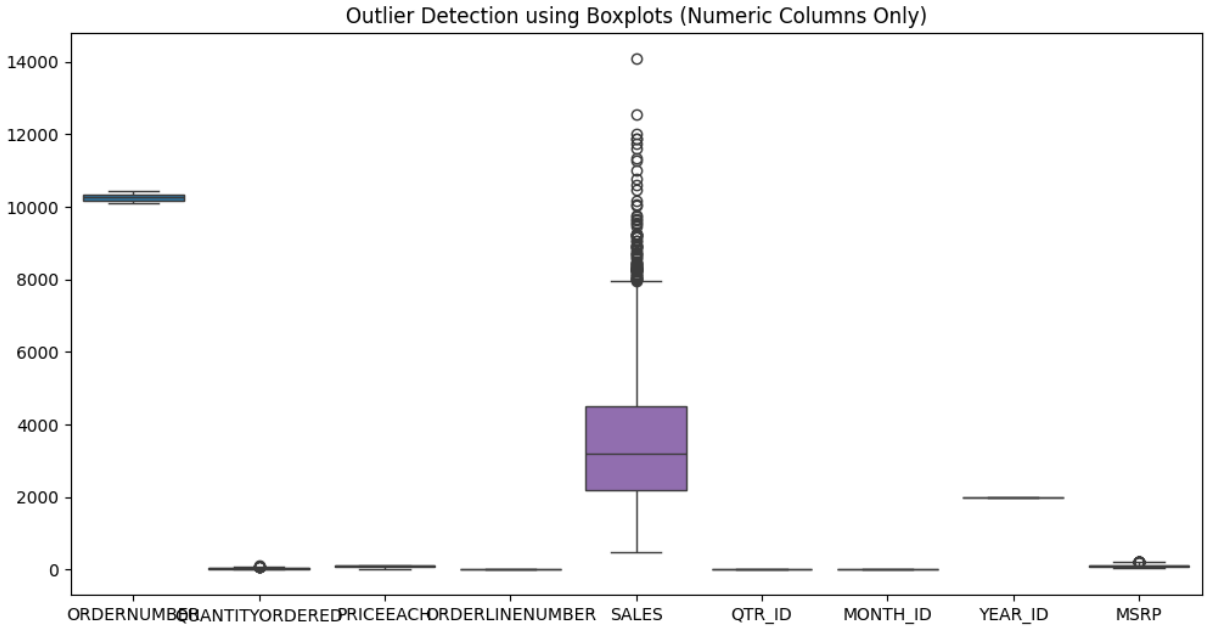
# Visualize outliers
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 6))
sns.boxplot(data=df_numeric)
plt.title("Outlier Detection using Boxplots (Numeric Columns Only)")
```

```
plt.show()

# Identify outliers using IQR
Q1 = df_numeric.quantile(0.25)
Q3 = df_numeric.quantile(0.75)
IQR = Q3 - Q1

# Count outliers per column
outliers = ((df_numeric < (Q1 - 1.5 * IQR)) | (df_numeric > (Q3 + 1.5 * IQR))).sum(
print("\nNumber of Outliers per Numeric Feature:\n", outliers)
```




```
Number of Outliers per Numeric Feature:
 ORDERNUMBER          0
QUANTITYORDERED      8
PRICEEACH            0
ORDERLINENUMBER      0
SALES               81
QTR_ID               0
MONTH_ID             0
YEAR_ID              0
MSRP                28
dtype: int64
```

In [15]:

```
# Select numeric columns only
df_numeric = df.select_dtypes(include=['int64', 'float64'])

print(df_numeric.head())
```

```
   ORDERNUMBER  QUANTITYORDERED  PRICEEACH  ORDERLINENUMBER    SALES  QTR_ID  \
0        10107               30      95.70                2  2871.00       1
1        10121               34      81.35                5  2765.90       2
2        10134               41      94.74                2  3884.34       3
3        10145               45      83.26                6  3746.70       3
4        10159               49     100.00               14  5205.27       4

   MONTH_ID  YEAR_ID  MSRP
0         2     2003    95
1         5     2003    95
2         7     2003    95
3         8     2003    95
4        10     2003    95
```

In [16]:
```python
# Calculate Q1, Q3 and IQR
Q1 = df_numeric.quantile(0.25)
Q3 = df_numeric.quantile(0.75)
IQR = Q3 - Q1

# Calculate lower and upper limits
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

# Cap outliers
df_capped = df_numeric.clip(
    lower=lower_limit,
    upper=upper_limit,
    axis=1
)

print("Outliers handled using IQR capping.")
print(df_capped.shape)
```

```
Outliers handled using IQR capping.
(2823, 9)
```

In [17]:
```python
# Normalize data
scaler = StandardScaler()

X_scaled = scaler.fit_transform(df_capped)

print("Data normalized using StandardScaler.")
```

```
Data normalized using StandardScaler.
```

In [18]:
```python
df_normalized = pd.DataFrame(X_scaled, columns=df_capped.columns)

print("\nSample of Normalized Data:")
display(df_normalized.head())

print("\nMean of each feature after normalization:\n", df_normalized.mean())
print("\nStandard deviation of each feature after normalization:\n", df_normalized.
```

```
Sample of Normalized Data:
```

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	QTR
0	-1.647947	-0.526907	0.596978	-1.057059	-0.374413	-1.427(
1	-1.495888	-0.110427	-0.114450	-0.347015	-0.435116	-0.596(
2	-1.354689	0.618413	0.549384	-1.057059	0.210870	0.234(
3	-1.235214	1.034894	-0.019759	-0.110334	0.131372	0.234(
4	-1.083154	1.451374	0.810158	1.783116	0.973809	1.065(

```
Mean of each feature after normalization:
 ORDERNUMBER       -5.927482e-15
QUANTITYORDERED   -2.139431e-17
PRICEEACH         -3.221731e-16
ORDERLINENUMBER    9.312817e-17
SALES              2.416298e-16
QTR_ID             7.550932e-17
MONTH_ID          -1.761884e-17
YEAR_ID           -1.988412e-15
MSRP               1.208149e-16
dtype: float64

Standard deviation of each feature after normalization:
 ORDERNUMBER        1.000177
QUANTITYORDERED    1.000177
PRICEEACH          1.000177
ORDERLINENUMBER    1.000177
SALES              1.000177
QTR_ID             1.000177
MONTH_ID           1.000177
YEAR_ID            1.000177
MSRP               1.000177
dtype: float64
```

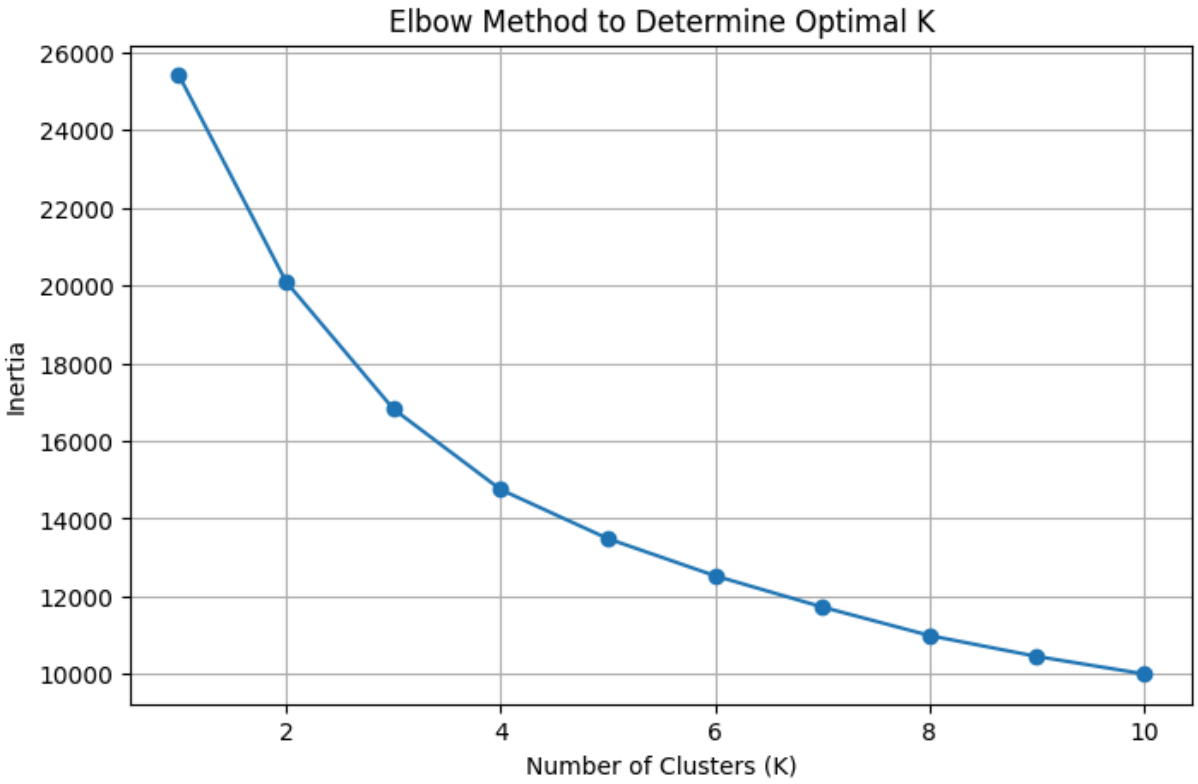
In [19]:

```python
inertia = []
K = range(1, 11)

for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

# Plot the Elbow curve
plt.figure(figsize=(8, 5))
plt.plot(K, inertia, marker='o')
plt.title("Elbow Method to Determine Optimal K")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.grid(True)
plt.show()
```

```
C:\Users\HP\AppData\Roaming\Python\Python313\site-packages\joblib\externals\loky\bac
kend\context.py:131: UserWarning: Could not find the number of physical cores for th
e following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by setti
ng LOKY_MAX_CPU_COUNT to the number of cores you want to use.
  warnings.warn(
  File "C:\Users\HP\AppData\Roaming\Python\Python313\site-packages\joblib\externals
\loky\backend\context.py", line 247, in _count_physical_cores
    cpu_count_physical = _count_physical_cores_win32()
  File "C:\Users\HP\AppData\Roaming\Python\Python313\site-packages\joblib\externals
\loky\backend\context.py", line 299, in _count_physical_cores_win32
    cpu_info = subprocess.run(
        "wmic CPU Get NumberOfCores /Format:csv".split(),
        capture_output=True,
        text=True,
    )
  File "C:\ProgramData\anaconda3\Lib\subprocess.py", line 554, in run
    with Popen(*popenargs, **kwargs) as process:
         ~~~~~^^^^^^^^^^^^^^^^^^^^^
  File "C:\ProgramData\anaconda3\Lib\subprocess.py", line 1039, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
    ~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                        pass_fds, cwd, env,
                        ^^^^^^^^^^^^^^^^^^^
    ...<5 lines>...
                        gid, gids, uid, umask,
                        ^^^^^^^^^^^^^^^^^^^^^^
                        start_new_session, process_group)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\ProgramData\anaconda3\Lib\subprocess.py", line 1554, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
                       ~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
                             # no special security
                             ^^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
                             cwd,
                             ^^^^
                             startupinfo)
                             ^^^^^^^^^^^^
```



In [20]:
```
from sklearn.metrics import silhouette_score

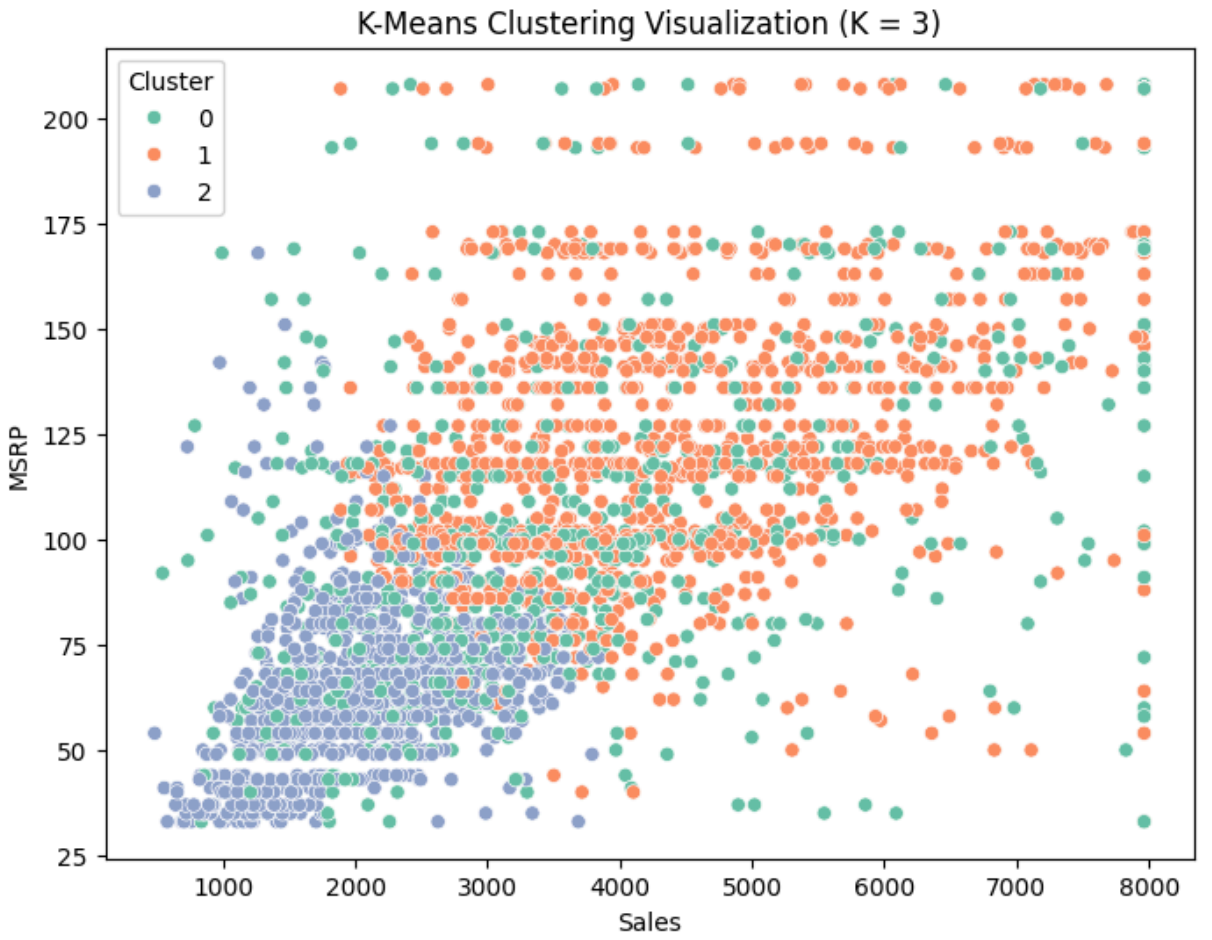
for k in range (2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    sil_score = silhouette_score(X_scaled, labels)
    print(f"K = {k} → Silhouette Score = {sil_score:.4f}")
```

```
K = 2 → Silhouette Score = 0.2154
K = 3 → Silhouette Score = 0.1976
K = 4 → Silhouette Score = 0.2077
K = 5 → Silhouette Score = 0.1874
K = 6 → Silhouette Score = 0.1938
K = 7 → Silhouette Score = 0.1910
K = 8 → Silhouette Score = 0.1851
K = 9 → Silhouette Score = 0.1843
K = 10 → Silhouette Score = 0.1864
```

In [21]:
```
#Visualize Clusters for K = 3
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)

plt.figure(figsize=(8,6))
sns.scatterplot(
    x=df_capped['SALES'],
    y=df_capped['MSRP'],
    hue=labels,
    palette='Set2'
)
plt.title("K-Means Clustering Visualization (K = 3)")
plt.xlabel("Sales")
plt.ylabel("MSRP")
```

```
plt.legend(title='Cluster')
plt.show()
```



In [ ]: